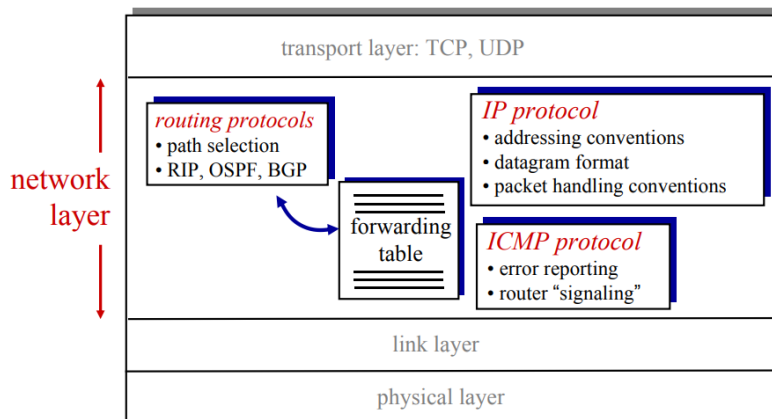


# lecture 11 (Network Layer)

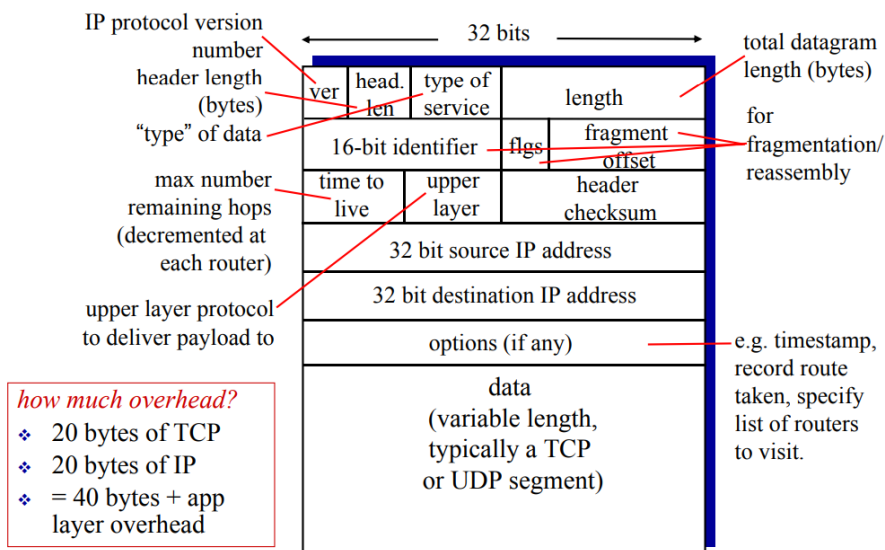
- IP : Internet Protocol



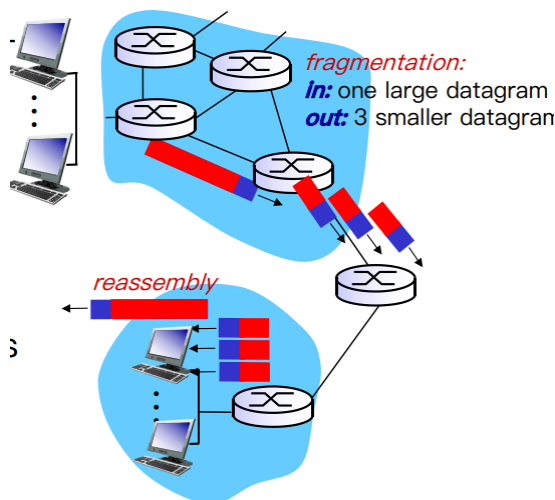
- 主机与路由器的网络层功能：路由协议负责路径选择（例如 RIP、OSPF、BGP），这些协议计算并更新用于生成转发表的路由信息。
  - 说明：路由协议属于控制平面，用来决定“应该怎样走”——它们把路径信息下发到转发表，使路由器知道下一跳。
- 转发表（forwarding table）：转发表保存目的地址到输出接口的映射，数据包到达时用它进行逐包转发。
  - 说明：转发表是数据平面的核心，路由器查这个表决定把包送到哪个接口，查表动作必须很快以实现线速转发。
- IP 协议（IP protocol）定义寻址约定、数据报格式与包的处理约定，这些规范决定了 IP 报头的语义和路由器如何处理数据报。
  - 说明：IP 是网络层的实际协议，它规定地址怎样表示、数据报怎样封装及路由器应如何解析与转发包。
- ICMP 协议（ICMP protocol）用于错误报告与路由器之间的“信令”（例如目的不可达、超时、诊断消息）。
  - 说明：ICMP 是用来给主机或管理系统反馈网络异常或诊断信息的工具，不用于数据传输本身但对网络运维很重要。

- IP 数据报格式（IP datagram format）

# IP datagram format



- IP 头含有版本号、头长、服务类型等字段，版本指明 IPv4/IPv6，头长表示 IP 头的字节数，服务类型用于差异化服务。
  - 说明：这些字段告诉接收方/路由器如何解析头部与按何种优先级处理包，头长用于定位后续字段与有效载荷。
- 长度字段（total datagram length）表示整个数据报的字节总长度（包括 IP 头和数据），用于路由器与接收端用长度字段确定数据报边界与是否接收完整数据。
- 16 位标识符与标志位、片偏移（fragment offset）用于分片与重组：标识符把同一个原始报文的所有片关联起来，偏移与标志位指示片在原始报文中的位置与是否还有后续片。
  - 说明：当路由器因链路 MTU 而把一个大数据报拆成多个片时，这些字段帮助目的端把它们按顺序拼回去；偏移以 8 字节为单位计数。
- 生存时间（TTL, time to live）字段在每经过一个路由器时减一，用于防止包在网络中无限循环，当变为 0 时丢弃并可发送 ICMP 超时报文。
  - 说明：TTL 是环路保护机制，防止路由环路导致包永远在网内流转。
- 上层协议字段（upper layer）指明包的载荷应该交给哪个传输层协议（如 TCP 或 UDP），头部校验和用于检测 IP 头在传输中是否出错。
  - 说明：路由器可能只校验 IP 头的完整性并转发；接收端使用上层协议字段把有效载荷交给正确的传输协议处理。
- 源地址和目的地址各 32 位，选项（若存在）可承载时间戳、记录路径等可选功能；数据部分为可变长度，通常为一个 TCP 或 UDP 段。
  - 说明：地址字段用于路由查找与转发；选项很少用但用于诊断/控制；数据则是上层实际传输的负载。
- 协议开销举例：典型地 TCP 头 20 字节 + IP 头 20 字节 = 40 字节（还要加应用层的开销）。
  - 说明：这有助计算有效载荷比例（效率），在高频短报文场景头部开销会较高影响效率。
- IP 分片与重组（IP fragmentation, reassembly）
  -



- 链路有最大传输单元 MTU（最大可能的链路层帧），不同链路类型有不同的 MTU，因此需要考虑跨链路的最大帧。
  - 说明：MTU 决定了在某条链路上单个链路层帧能装多少字节的 IP 数据报（包括头部），超过 MTU 必须分片或在源端通过 Path MTU Discovery 限制发送大小。
- 大的 IP 数据报在网络中被分割（“分片”）成若干小的数据报，重组只在最终目的地进行。
  - 说明：中间路由器可能根据下游链路 MTU 把一个大数据报拆分成多个片，目的机负责把片按 ID 与偏移组合回原始数据报；中间路由器通常不重组。
- IP 头中的字段（标识符、偏移、标志）用于标识和按顺序重组相关片段。
  - 说明：这些字段确保目的机能把正确的片匹配在一起，并知道片在原始数据报中的位置以及是否还有后续片。
- 分片示例

## IP fragmentation, reassembly

*example:*

- ❖ 4000 byte datagram
- ❖ MTU = 1500 bytes

1480 bytes in data field

offset =  $1480/8$

| length | ID | fragflag | offset |
|--------|----|----------|--------|
| =4000  | =x | =0       | =0     |

*one large datagram becomes several smaller datagrams*

| length | ID | fragflag | offset |
|--------|----|----------|--------|
| =1500  | =x | =1       | =0     |
| =1500  | =x | =1       | =185   |
| =1040  | =x | =0       | =370   |

- 原始数据报长 4000 字节，某链路 MTU = 1500 字节，因此每片的可用数据字段是 1500 - IP 头（通常 20）= 1480 字节。

- 第一片：长度字段（total length）=1500，标识 ID = x，fragflag=1（还有更多片），offset=0（从最开始）。
- 第二片：长度=1500，ID = x，fragflag=1，offset = 1480/8 = 185（偏移以 8 字节为单位）。
  - 说明：第二片从原始数据第 1480 字节开始，因此偏移是 1480 除 8；依旧有后续片所以 fragflag=1。
- 第三片（尾片）：长度 = 剩余 1040 字节（含头部总长 1040），ID = x，fragflag = 0（没有后续片），offset = 370 (=2×185)。
- 说明：第三片是最后一片，fragflag=0 表示这是尾片；偏移表示它在原始报文中从 2×1480 字节处开始，目的端用这些信息重组成 4000 字节的完整数据。
- 说明与风险：所有片的 ID 必须相同以便目的端能识别属于同一原始数据报，若任一片丢失则目的端无法完成重组（通常会丢弃整个报文并上报错误或触发重传）。
  - 说明：分片增加了丢包敏感性（任何片丢失都使整个原始报文丢失），因此现实网络常尽量避免中间分片，改用源端的 Path MTU Discovery 来选择合适的分段大小。

## • IPv4 addressing

### • IP addressing

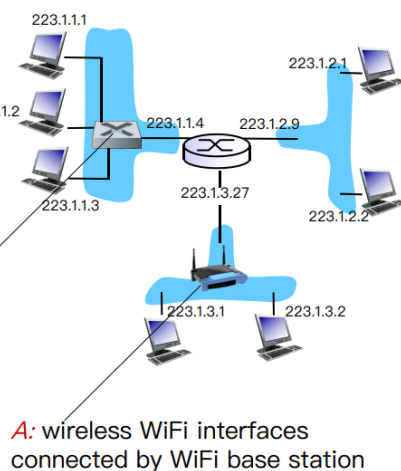
- IP 地址是 32 位的标识符，用来标识主机或路由器的每一个网络接口
  - IP 地址不是给“主机”一个全局唯一标签，而是给主机上的每个接口（网卡）分配地址，便于在不同网络上收发数据。
- 接口（interface）指网络接口卡，是主机/路由器与物理链路相连的位置
  - 路由器通常有多个接口（多个网口），主机通常只有一两个接口（有线以太网、无线 Wi-Fi 等），接口才是真正绑定 IP 的实体。
- 每个接口都有自己的 IP 地址
  - 因此同一台设备上可以有多个不同网段的地址——路由器就是通过这些不同接口把网段连接起来。

*Q: how are interfaces actually connected?*

*A: we'll learn about that in chapter 5, 6.*

*A: wired Ethernet interfaces connected by Ethernet switches*

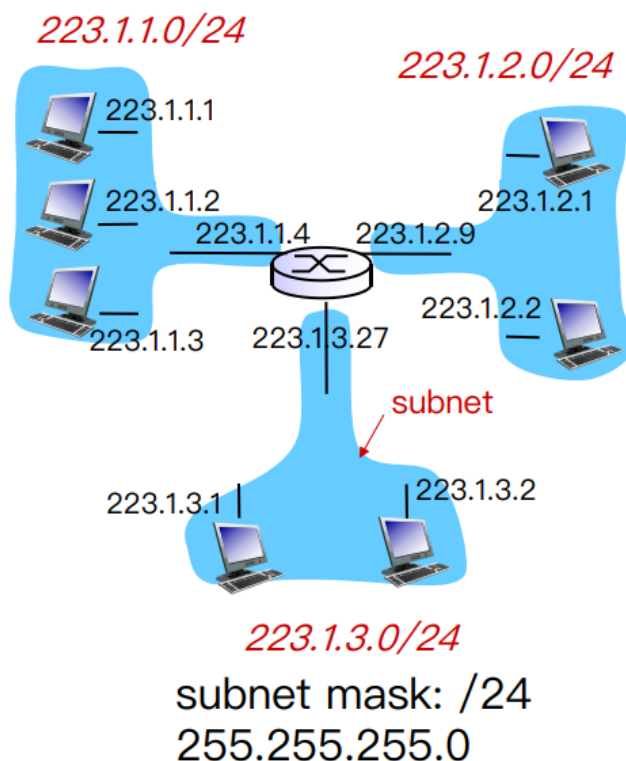
*For now: don't need to worry about how one interface is connected to another (with no intervening router)*



*A: wireless WiFi interfaces connected by WiFi base station*

- 有线以太网接口通过以太网交换机互联

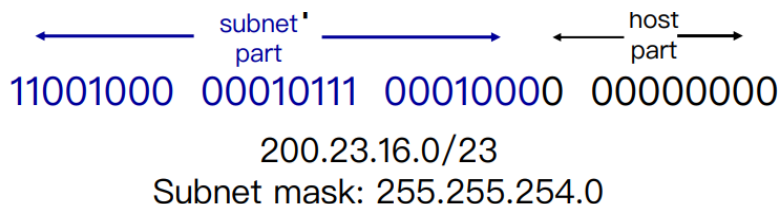
- 在局域网中，主机的有线网卡通常接到交换机端口，交换机再连到路由器或其它交换机形成网段。
- 无线 Wi-Fi 接口由无线基站（AP）连接
  - 无线设备通过接入点（AP）加入同一个子网，AP 与交换机/路由器相连实现对外通信。
- Subnet
  -



- IP 地址可分为子网部分（高位）和主机部分（低位）
- 具有相同子网位（即地址高位相同）的设备接口所组成的、可以彼此直接到达（无需通过路由器）的网络，就是子网；
  - 把网络按“能否直接通信”分组，就得到子网，每个子网在地址上共享相同的子网前缀。
- 示例：网络由 3 个子网构成（223.1.1.0/24、223.1.2.0/24、223.1.3.0/24），每个子网内的地址前缀相同
  - 解释：/24 表示前 24 位为子网位，常见掩码 255.255.255.0，便于计算与路由汇总。
- 要确定子网，可以把每个接口从其主机/路由器上分离出来，看哪些接口能直接相互到达，彼此形成的独立“岛”就是子网；
  - 从物理连接角度确定子网的方法——同一广播域或交换机下的接口通常属于同一子网。
- 子网是地址与物理连通性的结合体，是 IP 路由与转发的基本单位。
- How to assign/obtain IP address?

- **CIDR (无类域间路由)**：一种分配 IP 地址块的方法，允许子网部分具有任意长度。

- 地址格式：a.b.c.d/x，其中 x 是子网部分的比特数（前缀长度）。
  - 说明：例如 200.23.16.0/23 表示前 23 位是网络/子网位，剩下的位为主机位；写成点分十进制加斜线前缀是常用的表示法。



- ISP 获得地址块的方法：
  - **ICANN (Internet Corporation for Assigned Names and Numbers)** 负责分配地址、管理 DNS 并解决域名争议。
    - ICANN/区域注册机构 (RIR) 将地址大块分配给 ISP，ISP 再向下分配给客户或子网；这是全网地址管理的组织层次。
- 子网获得地址块的方法：
  - 从其上级 ISP 的地址空间中分配到一段连续地址。
    - 一个子网通常不是直接从 ICANN 得到地址，而是 ISP 把自己得到的地址块按需要再划分、分配给下游组织或子网。
  - 示例：ISP 的大块地址按小块分给若干组织 (Organization 0、1、2 ...)，例如把 200.23.16.0/20 划分为多个 /23 给不同组织

|                |                                     |                |
|----------------|-------------------------------------|----------------|
| ISP's block    | 11001000 00010111 00010000 00000000 | 200.23.16.0/20 |
| Organization 0 | 11001000 00010111 00010000 00000000 | 200.23.16.0/23 |
| Organization 1 | 11001000 00010111 00010010 00000000 | 200.23.18.0/23 |
| Organization 2 | 11001000 00010111 00010100 00000000 | 200.23.20.0/23 |
| ...            | ....                                | ....           |
| Organization 7 | 11001000 00010111 00011110 00000000 | 200.23.30.0/23 |

- 

Suppose all of the interfaces in each of these three subnets are required to have the prefix 166.4.20.128/25.

- Subnet 1 is required to support at least 62 interfaces 166.4.20.128/26 (64个地址)
- Subnet 2 is required to support at least 30 interfaces 166.4.20.192/27 (32个地址)
- Subnet 3 is required to support at least 28 interfaces 166.4.20.224/27 (32个地址)

Provide three network addresses (of the form a.b.c.d/x) that satisfy these constraints.

Block of addresses: 166.4.20.128/25. Subnet 1: at least 62 interfaces; Subnet 2: at least 30 interfaces; Subnet 3: at least 28 interfaces

This block of IP addresses can be written as

10100110 00000100 00010100 10000000

Subnet 1: since  $2^6 = 64 > 62$ , we can assign the following block

10100110 00000100 00010100 10000000

which can be represented as 166.4.20.128/26.

Subnet 2: since  $2^5 = 32 > 30$ , we can assign the following block

10100110 00000100 00010100 11000000

which can be represented as 166.4.20.192/27.

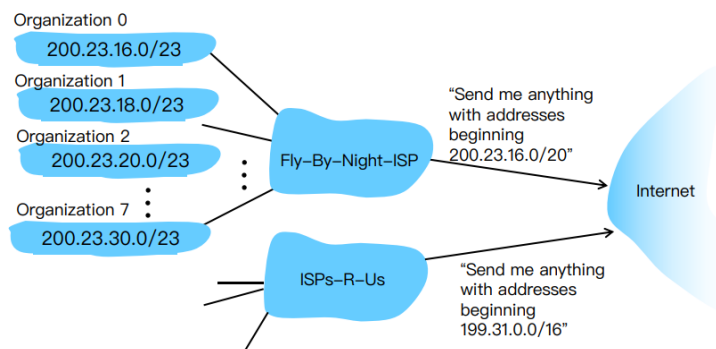
Subnet 3: since  $2^5 = 32 > 38$ , we can assign the following block

10100110 00000100 00010100 11100000

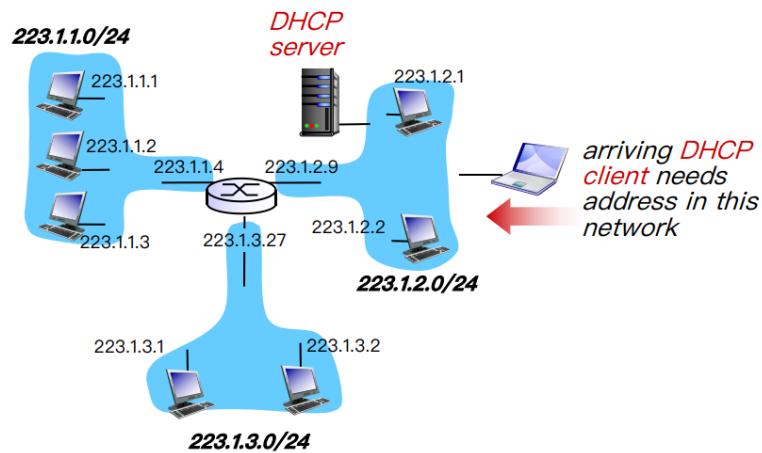
which can be represented as 166.4.20.224/27.

- 分层/聚合地址用于路由：

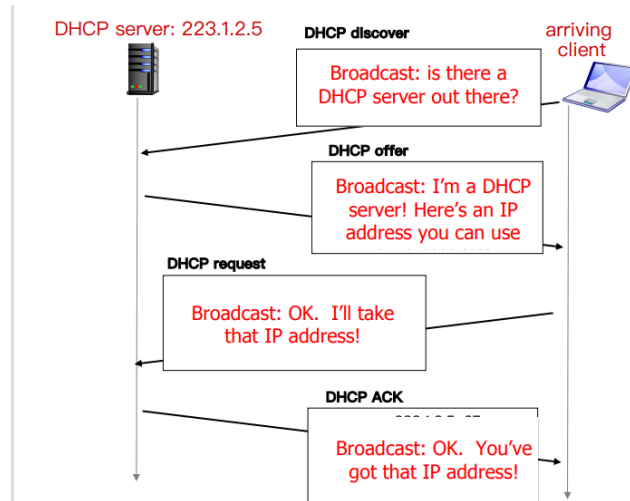
- 分层地址允许 ISP 将多段连续的 /23 汇总成更大的前缀（例如把若干 200.23.16.0/23、200.23.18.0/23... 汇总为 200.23.16.0/20）并向上游只宣告一个聚合路由，从而减少全网路由条目数。



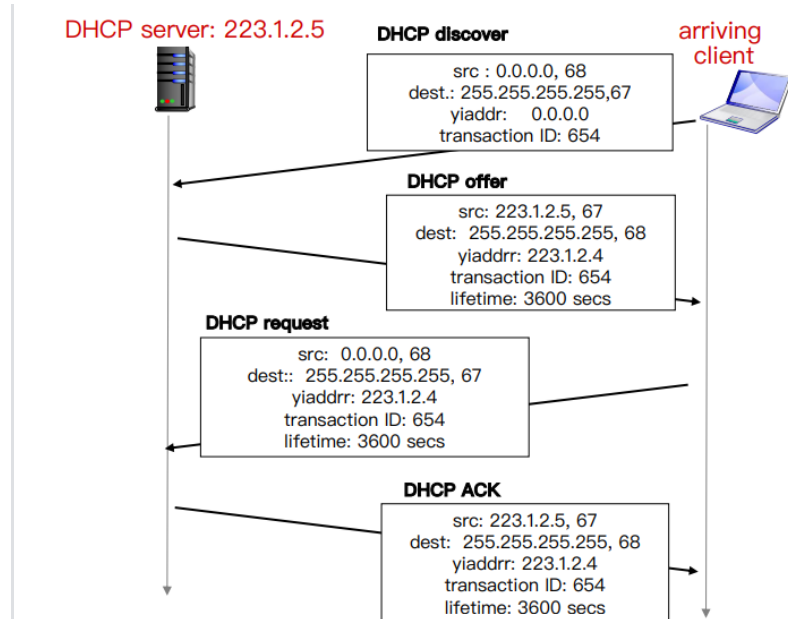
- 路由聚合利用地址的层次性（ISP 拥有一大块，再分配小块给客户），这样上游只需一条汇总路由就能到达 ISP，显著提高路由表和路由通告的效率。
- 主机获得 IP 的方法：
  - 主机可以由系统管理员在配置文件中手动设置固定地址（例如 Windows 控制面板或 UNIX /etc/rc.config）
    - 利于长期稳定配置与服务器
  - 也可以使用 **DHCP（动态主机配置协议）** 由服务器动态获取地址，实现即插即用。
    - DHCP 则方便大量终端自动配置、移动设备取地址并得到网关/DNS 等参数，常用于局域网。
    - DHCP 客户端—服务器场景：
      - 图示：在多个子网（223.1.1.0/24、223.1.2.0/24、223.1.3.0/24）中，当有新到的客户端需要在其所在子网获得地址时，客户端向本地广播 DHCP 请求，DHCP 服务器（本地或通过中继）回应并分配一个属于该子网的地址。



- 若服务器不在本地子网则依赖 DHCP 中继转发消息以保证客户端能拿到正确子网内的地址与配置信息。
- DHCP 目标：允许主机在加入网络时从网络服务器动态获取其 IP 地址
  - 支持续租（renew）现有租用地址
    - DHCP 用“租用”机制避免地址长期浪费，同时方便笔记本/手机等设备随插即用地址获得网络参数。
  - 允许地址重用（只在连接/开机时占用地址）
  - 对移动用户尤为有用。
- DHCP 概览（四步）：
  - 主机广播发送“DHCP discover”消息寻找服务器
  - DHCP 服务器用“DHCP offer”广播回应可用地址
  - 主机用“DHCP request”请求所选地址
  - 服务器确认并用“DHCP ack”完成分配，完成后主机得到地址、子网掩码、网关、DNS 等配置
- DHCP 客户端—服务器示意流程：
  - 到达的客户端先广播 DHCPDISCOVER（“网络上有 DHCP 服务器吗？”），服务器广播 DHCPOFFER（“我是 DHCP 服务器，给你这个地址”），客户端广播 DHCPREQUEST（“我接受这个地址”），服务器广播 DHCPACK（“确定，你有这个地址”）。



- 图中每一步通常使用广播（本地子网广播）以便即便客户端还没有地址也能接收/发送发现与请求消息；若服务器不在本地，通常通过 DHCP 中继转发这些消息。
- DHCP 报文示例与字段：
  - DHCPDISCOVER 中源是 0.0.0.0、目的 255.255.255.255，包含事务 ID；DHCPOFFER、DHCPREQUEST、DHCPACK 的报文字段包含建议的 yiaddr（被分配的地址）、服务器标识、事务 ID 和租期（lifetime）等信息。



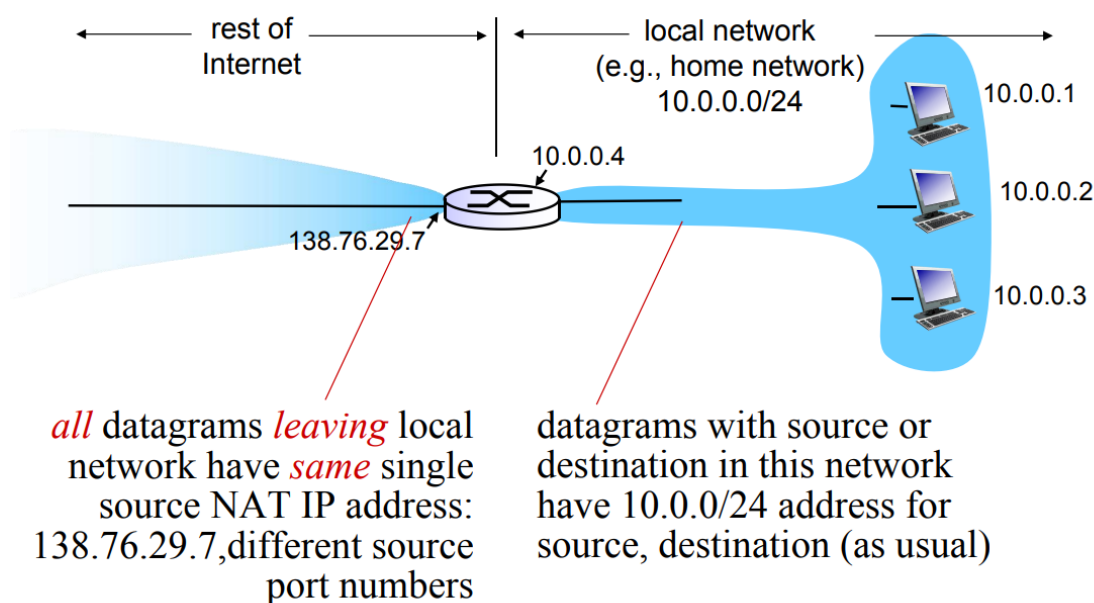
- 事务 ID 关联同一会话的消息，yiaddr 表示“your IP address”，租期字段指示客户端可使用该地址的时间长度（过期需续租）。
- DHCP：不只分配 IP 地址：
  - DHCP 不仅返回分配好的 IP 地址，还可返回子网的第一跳路由器地址（默认网关）、DNS 服务器的名称与 IP、以及网络掩码（指示网络/主机部分），从而一次性把主机所需的关键网络参数下发。
  - 因此 DHCP 常用于把全部必要的网络配置（网关、DNS、掩码、租期等）一并下发，实现真正的“即插即用”。

## • 网络地址转换 network address translation (NAT)

- LANs 局域网（例如宿舍、学校实验室、校园或办公楼）需要一段地址范围由 ISP 分配来覆盖该 LAN 的所有 IP 设备，且如果子网变大就必须分配更大的地址块，说明了若每个 LAN 都要公网地址会导致地址需求巨大并难以扩展。
  - NAT 用途：把很多局域网内的私有地址映射到少量（甚至一个）公网地址上，从而节省公网 IPv4 地址并隐藏内网主机对外的真实地址。
- NAT 的基本做法是让局域网使用保留的私有地址范围，不直接占用公网地址池。

| RFC 1918 name | IP address range              | Number of addresses | Largest CIDR block (subnet mask) | Host ID size | Mask bits | Classful description <sup>[Note 1]</sup> |
|---------------|-------------------------------|---------------------|----------------------------------|--------------|-----------|------------------------------------------|
| 24-bit block  | 10.0.0.0 – 10.255.255.255     | 16 777 216          | 10.0.0.0/8 (255.0.0.0)           | 24 bits      | 8 bits    | single class A network                   |
| 20-bit block  | 172.16.0.0 – 172.31.255.255   | 1 048 576           | 172.16.0.0/12 (255.240.0.0)      | 20 bits      | 12 bits   | 16 contiguous class B networks           |
| 16-bit block  | 192.168.0.0 – 192.168.255.255 | 65 536              | 192.168.0.0/16 (255.255.0.0)     | 16 bits      | 16 bits   | 256 contiguous class C networks          |

- 私有地址只能在局域网内部使用，不能在 Internet 上做全局路由（需要 NAT 才能与公网通信）。
- NAT 示意图：私网到公网的地址转换

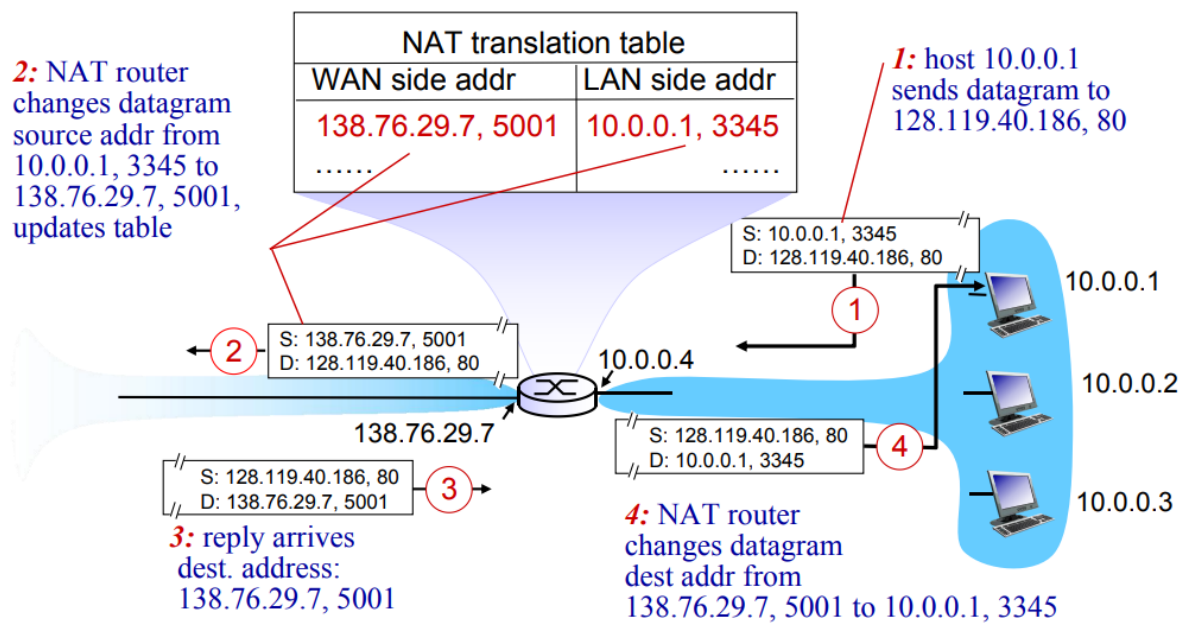


- 本地网络（例如 10.0.0.0/24）内的主机使用私有地址作为源/目的，而所有离开本地网络的数据包对外都用同一个公网 NAT 地址（例如 138.76.29.7）做源地址，同时通过不同的源端口号区分内部分机，目的端看到的都是相同的单一公网地址。
- 这是常用的PAT（端口地址转换）形式，多个内部主机共享一个公网 IP，路由器记录（IP,端口）映射以便回包能正确转回内网主机。
- 图中示例：本地网络（例如家庭网络）使用 10.0.0.0/24 子网，内部主机地址如 10.0.0.1、10.0.0.2、10.0.0.3。
  - 这些地址是私有的，不能直接在 Internet 上被路由或识别。
- 所有离开本地网络的数据报都使用相同的单一源 NAT 公网地址（示例为 138.76.29.7），但使用不同的源端口号加以区分。
  - 这是端口地址转换（PAT）的工作方式，多个内部连接复用一个公网 IP，通过端

口区分不同内部会话。

- 发往或来自该局域网的分组在内网仍使用 10.0.0.0/24 的地址作为源或目的（在本地通信中不变）。
  - 在本地网络内通信不经过 NAT，只有当数据要出到外部时，路由器才进行地址/端口替换。
- 动机：通过让本地网络对外只使用一个公网地址（例如示例中的 138.76.29.7），可以只用一个公网 IP 代表所有设备，从而能在不通知外部世界的情况下更改本地设备地址、在更换 ISP 时不必更改内网地址，并且内部设备对外界不直接可寻址。
- NAT 的动机 / 好处：
  - 动机：对外世界而言，本地网络只使用一个 IP 地址（例如示例中的 138.76.29.7）。
    - 这样可以显著减少对公网地址的消耗，并把内网隐藏在单一公网地址后面。
  - 只需为所有内网设备使用一个公网 IP。
  - 可以在不通知外部世界的情况下改变本地网络中设备的地址。
  - 可以在不更改内网设备地址的情况下更换 ISP。
    - 只需更换边缘 NAT/路由器的公网地址映射，内网配置无需变动，迁移更方便。
  - 局域网内部设备对外不可被直接寻址，在一定程度上增加了安全性。
    - NAT 把内网设备隐藏起来，外部主机无法直接发起到内网任意主机的连接（除非配置端口转发或 VPN），因此降低直接攻击面。
- NAT 的实现要点 / 行为要求：
  - 实现：NAT 路由器必须对外发的数据报做替换——把每个外发数据报的源 IP 地址和源端口替换为（NAT 的公网 IP，新端口），因此远端客户端/服务器会把回复发到该（NAT IP，新端口）。
  - 必须记住（在 NAT 翻译表中）每一个（源 IP 地址，端口号）到（NAT IP 地址，新端口号）的翻译对。
  - 对于入站（外部到内部）的数据报，要把目的字段中（NAT IP、新端口）替换为对应的（内部源 IP、端口）并转发，替换依据存储在 NAT 表中的对应关系。
- NAT 翻译过程示意图：
  -

# NAT: network address translation



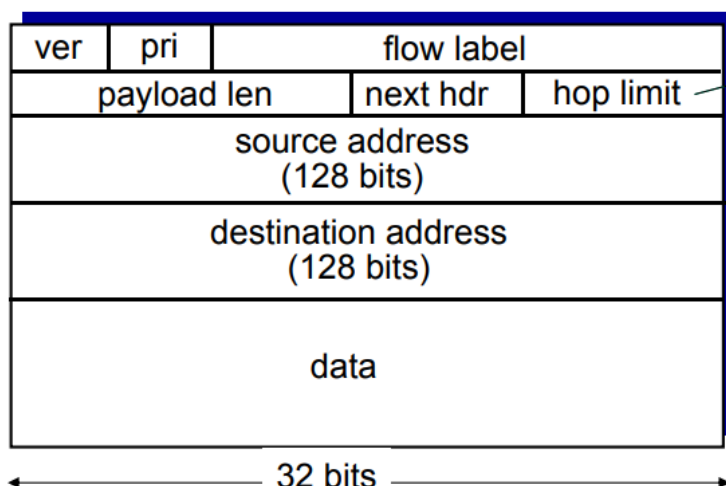
\* Check out the online interactive exercises for more examples: [http://gaia.cs.umass.edu/kurose\\_ross/interactive/](http://gaia.cs.umass.edu/kurose_ross/interactive/)

- NAT 翻译表（WAN 侧地址 | LAN 侧地址）。
  - NAT 路由器用一张表把外部（WAN）地址:端口和内部（LAN）地址:端口一一对应起来，作为回包转发的依据。
- 步骤 1：主机 10.0.0.1 向 128.119.40.186 的 80 端口发送数据报。
  - 这是内网主机发起对某外部服务器的连接请求（源 IP: 10.0.0.1，源端口 3345；目的服务器 IP: 128.119.40.186，目的端口 80）。
- 步骤 2：NAT 路由器把数据报的源地址从 10.0.0.1:3345 改成 138.76.29.7:5001，并在表中更新映射项。
- 步骤 3：远端服务器把数据发回到看到的源地址（NAT 的公网地址和端口），此时路由器收到该包并需查表。
- 步骤 4：NAT 路由器把到达的数据报目的地址从 138.76.29.7:5001 改回 10.0.0.1:3345，并转发到内网主机。
- NAT 的端口空间与争议点：
  - 16 位端口号字段：一个 LAN 侧地址可以支持约 60,000 个同时连接！
    - 理论上端口有 65535 个（减去保留端口等），因此通过端口复用（PAT）单个公网 IP 可同时承载数万条不同的会话。
  - NAT 有争议：路由器应该只处理到第 3 层（IP 层）转发，而 NAT 需要检查/修改第 4 层（端口）信息，增加了路由器的功能与复杂度。
  - NAT 违反端到端原则。
    - 端到端原则要求终端直接互通并在端点实现功能，NAT 在中间修改地址和端口，破坏了这种直接、透明的端到端通信模型。

- 因此应用设计者必须考虑 NAT 的可能性（例如 P2P 应用、服务器进程）。
  - 应用或服务若期望外网直接连接，必须实现 NAT 穿越策略或使用中继/映射（如 UPnP、STUN/TURN、端口转发、反向连接等）。
- 另一个注意点：知名端口（Well-known port number）。
  - 某些服务使用固定的“知名端口”（例如 80/443/22），在 NAT 环境下若要让外部发起连接到内网服务，通常需要静态一对一映射或端口转发来暴露这些端口。

## • IPv6

- IPv6 动机：
  - 初始动机：IPv4 地址不足，32 位地址空间很快会被完全分配完，需要更大的地址空间来容纳未来的大量设备。
  - 额外动机：新头格式有助于加快处理/转发。
    - IPv6 设计上简化并固定某些头字段（例如固定 40 字节首部），使路由器更容易快速解析并进行高效硬件转发。
- IPv6 数据报格式要点：
  - 128 位地址空间
  - 固定长度 40 字节头：定长头便于硬件快速处理
  - 不允许分片：把分片责任下沉到端点以提高中间路由器性能和简化转发路径（IPv6 用路径 MTU 探测避免中间分片）
- IPv6 数据报头字段与含义：



decremented by one by each router that forwards the Datagram; if reaches zero, the datagram is discarded.

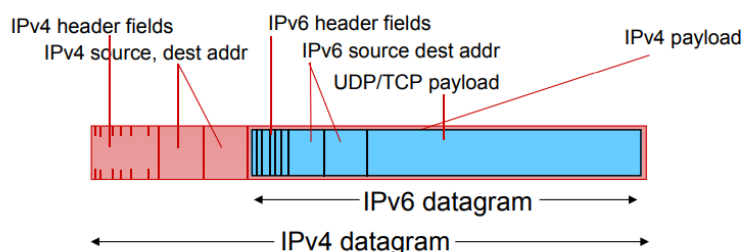
- priority（优先级）：用于 QoS/流量优先处理，供路由器或交换设备在必要时区分不同优先级流。
- flow label（流标签）：标识属于同一“流”的数据报。
- next header：标识上层协议（例如 TCP 或 UDP）。
  - 与 IPv4 的 protocol 字段类似，告诉接收端如何把载荷交给哪种传输层协议。
- IPv6 首部固定 40 字节（包含上述字段），hop limit（类似 IPv4 的 TTL）在每个转发路由器处减一，若减为 0 则丢弃数据报以防环路。

- 与 IPv4 的其它变化：

- 校验和 (checksum)：移除，路由器在转发时不再需要重新计算或验证包头的校验和，从而减少每跳的处理开销并提升转发性能。
  - 差错检测仍由链路层（例如以太网的 FCS）和传输层（如 TCP/UDP 的校验和）承担，所以端到端的数据完整性仍有保障。
- 选项 (options)：允许存在，但放在首部之外，由“下一首部 (Next Header)”字段指示。
  - IPv6 将原来可变长度、可能出现在主首部内的选项，改为放在可选的扩展首部 (extension headers) 中，通过 Next Header 字段链式指向。这使主首部保持固定和简洁，便于快速处理，同时仍能通过扩展首部支持额外功能。
- 不进行分片 (no fragmentation)：如果数据报过大以致无法在出接口转发，路由器会直接丢弃该数据报，并发送“数据包过大 (Packet Too Big)”的 ICMP 错误消息。
  - 在 IPv6 中，路由器不对经过的数据包做分片。源主机负责根据路径 MTU (Path MTU) 进行分片或减小报文大小（通过路径 MTU 发现机制）。当路由器遇到超出下一跳链路 MTU 的包时，路由器会丢弃并返回 ICMPv6 的 Packet Too Big，包内通常包含可用的 MTU 信息，提示发送端调整。这简化了路由器实现并避免中间复合分片带来的重组开销，但把责任交给了端点进行 MTU 管理。

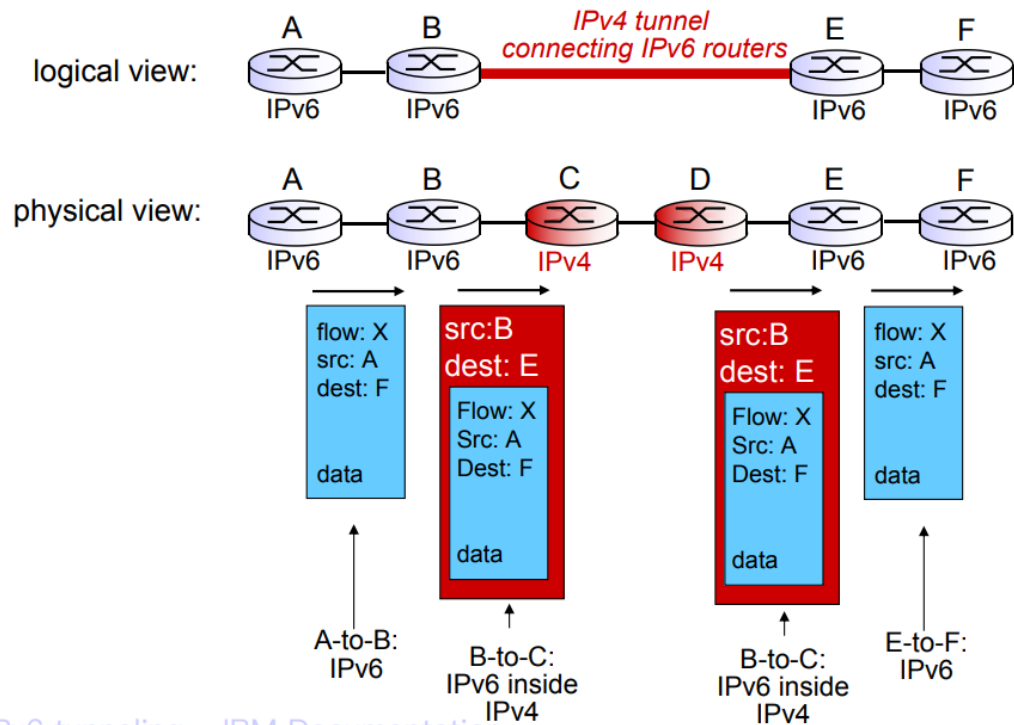
- 从 IPv4 迁移到 IPv6：

- 因为全球大量设备和运营商难以在同一时间全部升级，故不存在一次性全部切换的场景。
  - 必须采用过渡机制保证 IPv4/IPv6 混合环境下的互通和逐步部署。
- 隧道 (tunneling)：在 IPv4 路由器之间，将 IPv6 数据报作为 IPv4 数据报的负载承载。



- 解释：当两端（或若干端）是 IPv6 网络/路由器、但中间传输路径只支持 IPv4 时，可以把完整的 IPv6 数据报封装 (encapsulate) 到一个 IPv4 数据报的负载里，发送到对端的隧道端点，由对端解封装 (decapsulate) 后继续原来的 IPv6 转发。隧道端点负责对外使用 IPv4 转发、对内保持 IPv6。优点是可以在现有 IPv4 基础设施上连接 IPv6 “孤岛”；缺点有额外开销、MTU 问题和性能影响。
- 隧道示意：逻辑视图与物理视图

# Tunneling

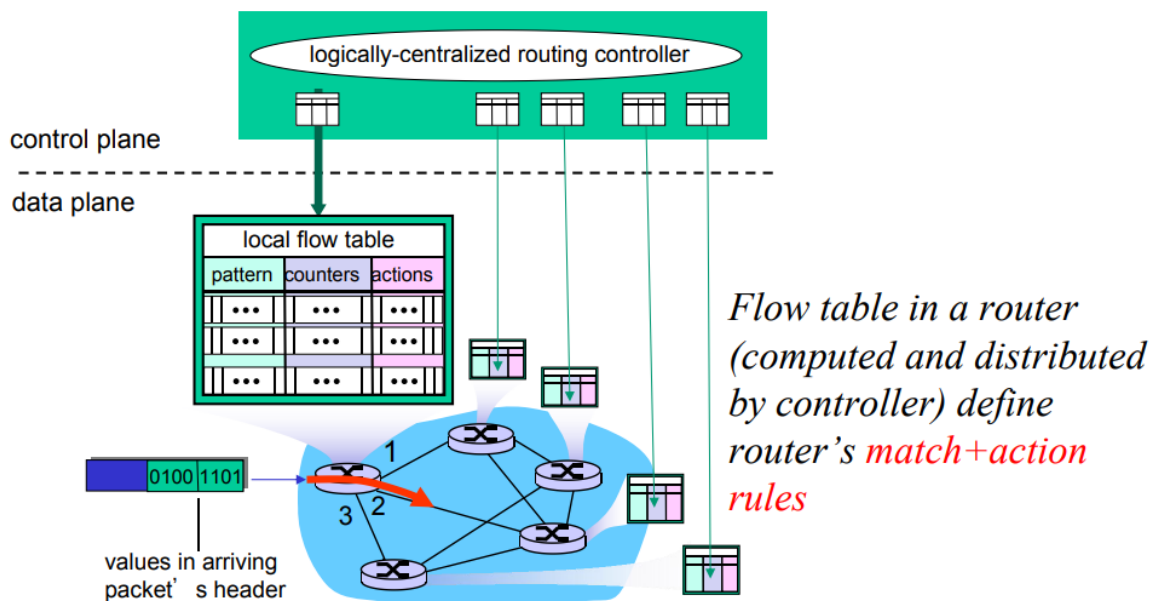


## [IPv6 tunneling – IBM Documentation](#)

- 逻辑视图 (logical view)：在逻辑上，A-B 和 E-F 是通过一个 IPv4 隧道直接连接的 IPv6 路由器链路
- 物理视图 (physical view)：实际物理路径上在 B 与 E 之间存在仅支持 IPv4 的中间路由器（例如 C、D），于是 B 将 IPv6 数据报封装到 IPv4 中发送（B 到 C），C、D 按 IPv4 路由转发，直到到达 E，E 再解封装恢复 IPv6。
- 解释步骤（分点）：
  - A→B：在 A 到 B 的本地链路上直接发送 IPv6 数据报（正常 IPv6 转发）。
  - B→C（封装）：B 将完整的 IPv6 数据报放进一个 IPv4 数据报的负载中（外层 IPv4 首部的源地址设为 B，目的地址设为 E），然后作为普通 IPv4 报文发出。
  - C、D：中间的 IPv4 路由器按外层 IPv4 首部进行转发，不需要理解内部的 IPv6。
  - E（解封装）→F：E 收到后去掉外层 IPv4 首部，恢复内部的 IPv6 数据报并把它在 IPv6 网络中转发到 F。
  - 内、外首部关系：内部保留原始 IPv6 的源/目的（src:A dest:F），外层 IPv4 首部仅用于在 IPv4 网络中将封装数据送到隧道对端（src:B dest:E）。
- 说明与注意事项：隧道可以是静态配置或自动隧道；需要注意封装带来的 MTU 降低（可能引起 Packet Too Big），以及隧道端点的处理性能、安全（封装绕过某些过滤器）等问题。

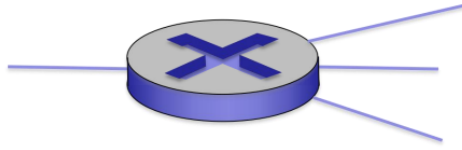
## • Generalized Forward and SDN

- 广义转发 & 软件定义网络（SDN）



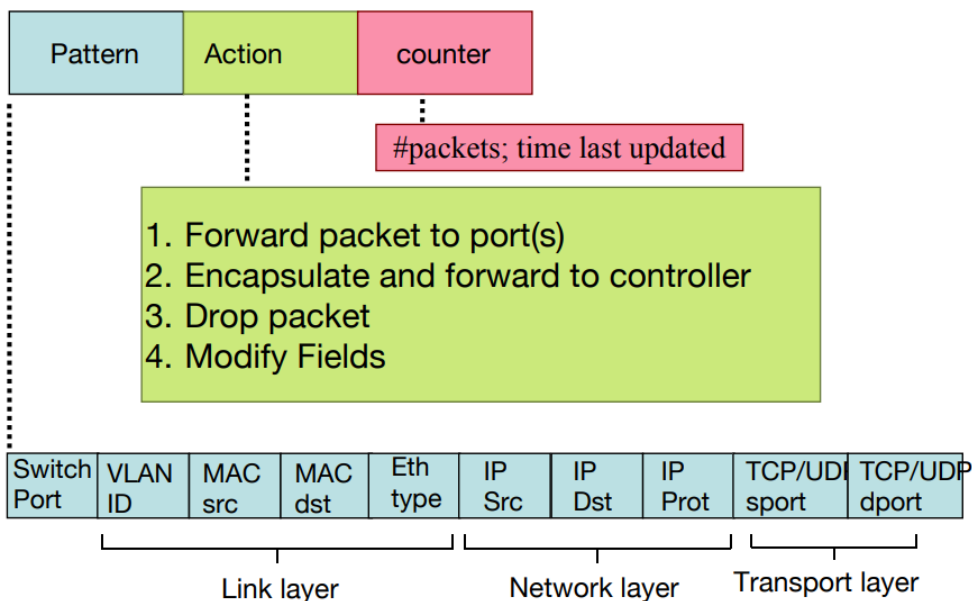
- 强调控制逻辑与转发逻辑的分离。
- 每个路由器包含一个流表（flow table），该流表由一个逻辑上中心化的路由控制器计算并分发。
  - 流表存放匹配规则与对应动作，控制器在控制平面集中计算策略，然后把表项下发到各个设备的数据平面；“逻辑上中心化”指控制器在逻辑上是单一决策点，物理上可以是分布式的以提高可用性和扩展性。
- 图示（control plane / data plane）：
  - 控制平面（control plane）在上层，由中心化控制器维护和下发规则。
  - 数据平面（data plane）在下层，每个交换/路由设备本地保存“本地流表（local flow table）”，流表包含：模式（pattern）、计数（counters）、动作（actions）。
    - 解释：模式用于匹配到来的报文头字段，计数用于统计匹配流量（例如包数、最后更新时间），动作定义匹配后如何处理（转发、丢弃、修改、发给控制器等）。
- 标注：“values in arriving packet's header”（到达数据包首部的值）。
  - 数据平面通过检查到达包的头字段值来与流表中的模式比对，找到匹配的表项并执行相应动作。
- 路由器中的流表（由控制器计算并分发）定义路由器的“匹配 + 动作”规则。
  - 这强调流表是路由器行为的核心，控制器负责策略决策，设备执行匹配与动作。
- OpenFlow 数据平面：
  - 广义转发：简单的数据包处理规则。
  - Pattern（模式）：匹配包首部字段中的值，例如匹配 IP 源/目的、MAC、端口号等字段。
  - Actions（对匹配包的动作）：对匹配到的包执行的操作包括：丢弃（drop）、转发（forward）、修改（modify）、将匹配的数据包或其副本发送给控制器（send matched packet to controller）。

- 控制器可以收到包样本来决定新的流表规则或做更复杂处理；也可以下发直接的转发/修改指令以实现策略。
- Priority（优先级）：用于消除重叠模式的歧义。
- Counters（计数器）：记录包的数量以及最后更新时间等统计信息，用于流量监控、超时、策略调整等。
- 



1. src=1.2.\*.\*, dest=3.4.5.\* → drop  
 2. src = \*.\*.\*, dest=3.4.\*.\* → forward(2)  
 3. src=10.1.2.3, dest=\*.\*.\* → send to controller

- OpenFlow：流表条目结构



- 描述单条表项包含的主要域与可执行的动作集合。
- 流表条目组成（从左到右）：
  - 模式（Pattern）
    - 匹配字段示例（底部，按协议层分类）：
      - 链路层（Link layer）：交换机端口（Switch Port）、VLAN ID、源 MAC（MAC src）、目的 MAC（MAC dst）、以太类型（Eth type）。
      - 网络层（Network layer）：IP 源（IP Src）、IP 目的（IP Dst）、IP 协议（IP Prot）。
      - 传输层（Transport layer）：TCP/UDP 源端口（TCP/UDP sport）、TCP/UDP 目的端口（TCP/UDP dport）。

- 解释：OpenFlow 允许匹配从链路层到传输层的多个字段组合，从而实现细粒度的流识别与控制。
- 动作（Action）
  - 可选动作举例（绿色大框）：
    - 将包转发到端口（Forward packet to port(s)）。
    - 封装并转发到控制器（Encapsulate and forward to controller）。
    - 丢弃包（Drop packet）。
    - 修改字段（Modify Fields）。
- 计数器（counter）
  - 计数器包含的信息示例：包的数量（#packets）；最后更新的时间（time last updated）。

- 基于目的地的转发（Destination-based forwarding）：

#### Destination-based forwarding:

| Switch Port | MAC src | MAC dst | Eth type | VLAN ID | IP Src | IP Dst   | IP Prot | TCP/UDP sport | TCP/UDP dport | Action |
|-------------|---------|---------|----------|---------|--------|----------|---------|---------------|---------------|--------|
| *           | *       | *       | *        | *       | *      | 51.6.0.8 | *       | *             | *             | port6  |

*IP datagrams destined to IP address 51.6.0.8 should be forwarded to router output port 6*

#### Destination-based layer 2 (switch) forwarding:

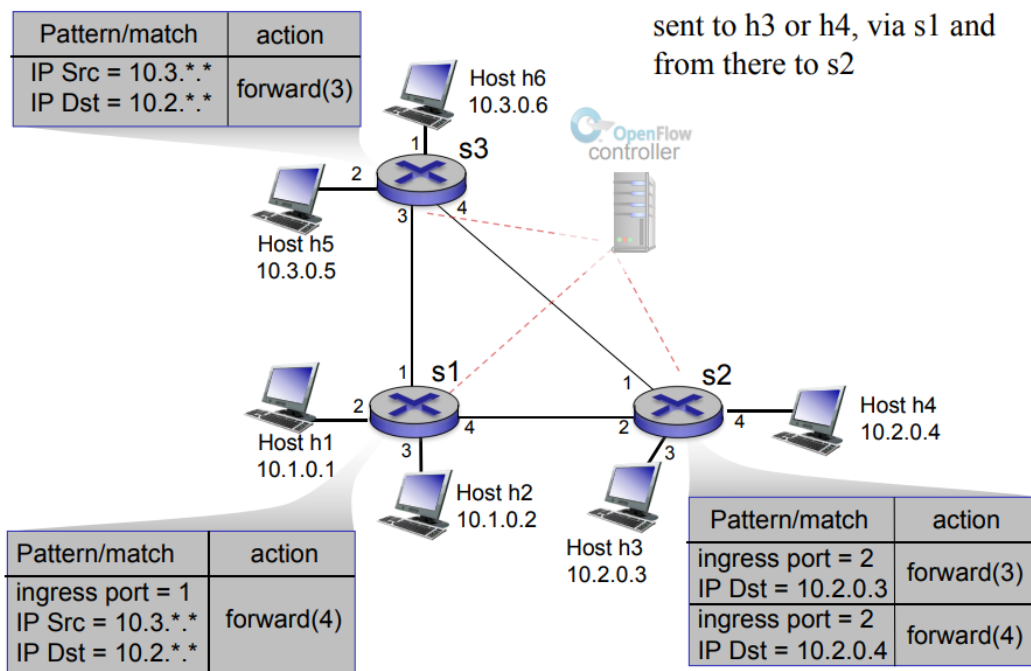
| Switch Port | MAC src           | MAC dst | Eth type | VLAN ID | IP Src | IP Dst | IP Prot | TCP/UDP sport | TCP/UDP dport | Action |
|-------------|-------------------|---------|----------|---------|--------|--------|---------|---------------|---------------|--------|
| *           | 22:A7:23:11:E1:02 | *       | *        | *       | *      | *      | *       | *             | *             | port3  |

*layer 2 frames from MAC address 22:A7:23:11:E1:02 should be forwarded to output port 6*

- 解释：讨论以目的地址为主要匹配条件的转发策略（路由器或交换机常用方式）。
- 基于目的地的转发：（表格展示各种字段，但强调只关心目的 IP）
  - IP 数据报目的地址为 51.6.0.8 的，应被转发到路由器的输出端口 6。
  - 常规路由表以目的 IP 决定下一跳或输出端口，其他字段用通配符（\*）表示不关心。
- 基于目的地的二层（交换）转发：（表格展示以 MAC 为主的匹配）
  - 来自 MAC 地址 22:A7:23:11:E1:02 的二层帧应被转发到输出端口（示例中标为某输出端口）。
  - 在二层交换中，通常根据目的 MAC（或来源 MAC 在学习交换中用于建立映射）决定转发端口；表项中对其它字段可使用通配符，重点匹配 MAC。

# OpenFlow example

*Example:* datagrams from hosts h5 and h6 should be sent to h3 or h4, via s1 and from there to s2



- 展示一个小网络拓扑与若干下发到交换机的流表条目，说明控制器如何通过流表控制转发路径。
- 来自主机 h5 和 h6 的数据报，应通过 s1 转发到主机 h3 或 h4（经由 s3 到 s1，再由 s1 到 s2）。
- Firewall — 防火墙

| Switch Port | MAC src | MAC dst | Eth type | VLAN ID | IP Src | IP Dst | IP Prot | TCP/UDP P | TCP/UDP dport | Forward |
|-------------|---------|---------|----------|---------|--------|--------|---------|-----------|---------------|---------|
| *           | *       | *       | *        | *       | *      | *      | *       | *         | 22            | drop    |

*do not forward (block) all datagrams destined to TCP port 22*

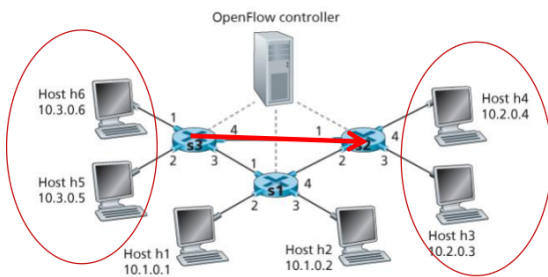
| Switch Port | MAC src | MAC dst | Eth type | VLAN ID | IP Src      | IP Dst | IP Prot | TCP/UDP P | TCP/UDP dport | Forward |
|-------------|---------|---------|----------|---------|-------------|--------|---------|-----------|---------------|---------|
| *           | *       | *       | *        | *       | 128.119.1.1 | *      | *       | *         | *             | drop    |

*do not forward (block) all datagrams sent by host 128.119.1.1*

- 用流表实现简单的包过滤策略（类似传统防火墙功能）。
- 上半部分规则：
  - 匹配字段：对所有字段使用通配符（\*），但是 TCP/UDP 目的端口 = 22
  - 丢弃（drop）
  - 该规则用于阻止 SSH（默认端口 22）等服务的外部访问；因为其他匹配位都为通配符，所以只要目的端口是 22 就会被丢弃。
- 下半部分规则：

- 匹配字段：对大部分字段使用通配符，但 IP 源 = 128.119.1.1
- 丢弃 (drop)
- 基于源地址的封锁，用于屏蔽某个恶意或不受信任的主机的所有流量。

#### • Firewall — 拓扑视图与 s2 的流表

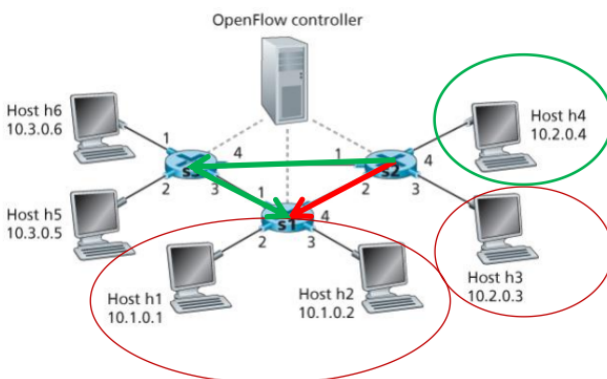


| s2 Flow Table (Example 3)           |            |
|-------------------------------------|------------|
| Match                               | Action     |
| IP Src = 10.3.*.* IP Dst = 10.2.0.3 | Forward(3) |
| IP Src = 10.3.*.* IP Dst = 10.2.0.4 | Forward(4) |

there were no other entries in s2's flow table

- 图中用红圈标出两个地址族/子网，一侧是源主机组（例如 10.3.x.x），另一侧是目的主机组（例如 10.2.x.x）；控制器在上方。
- s2 仅包含与该示例流量相关的条目，其余流量若无匹配会触发默认处理（例如发给控制器或丢弃，取决于策略）
  - 这强调流表可以非常针对性地仅为重要流量安装规则

#### • Load Balancing — 负载均衡示例：



| s2 Flow Table (Example 2)           |            |
|-------------------------------------|------------|
| Match                               | Action     |
| Ingress port = 3; IP Dst = 10.1.*.* | Forward(2) |
| Ingress port = 4; IP Dst = 10.1.*.* | Forward(1) |
| ...                                 | ...        |

- 展示如何用流表把流量在多个路径/服务器间分摊以实现负载均衡。

- 控制器可以根据实时负载信息定制转发策略，把到某一虚拟服务 IP 的流量引导到不同的具体主机或路径。
- 通过检查报文从哪一路径进入（入口端口），交换机可把来自不同来源的流量分别导向两个不同的下一跳或服务器，从而在不改变目的地址的前提下实现简单的流量分担。控制器可动态调整这些表项来响应负载变动。
- OpenFlow：
  - 这是对 OpenFlow 提供的数据平面抽象的总称，核心是把设备行为表示为“匹配 + 动作”的表项。
  - 无论是路由器、交换机、防火墙还是 NAT，都可以用“匹配字段 → 执行动作”的表项来表示，从而为控制器提供统一的编程接口。
  - Router（路由器）
    - 匹配：最长的目的地 IP 前缀
      - 传统路由器通过目的地址前缀最长匹配（最长前缀匹配）决定下一跳。
    - 动作：通过某条链路向外转发。
      - 匹配到具体路由表项后，路由器把报文转发到对应的输出接口/下一跳。
  - Switch（交换机）
    - 匹配：目的 MAC 地址。
      - 二层交换通常根据目的 MAC 去查表来决定转发端口。
    - 动作：转发。
      - 匹配到目的 MAC 后把帧发送到对应端口（或泛洪）。
  - Firewall（防火墙）
    - 匹配：IP 地址和 TCP/UDP 端口号。
      - 防火墙规则通常同时考虑地址和端口，以区分不同应用或服务。
    - 动作：允许或拒绝。
      - 匹配到规则后要么放行要么丢弃/拒绝，可能还做记录或告警。
  - NAT（网络地址转换）
    - 匹配：IP 地址和端口。
      - NAT 需要根据五元组（或至少地址+端口）识别连接流。
    - 动作：重写地址和端口。
      - 执行地址/端口转换（例如私有→公网），并可能记录映射以便返回流量正确反向转换。